A Fold Collapse in Two-Round Rainstorm-256 without Padding Compression

Cris
DOSAYGO Corporation
research@dosaygo.com

17 September 2026

Abstract

We give an algebraic collision reduction for a precisely specified variant of Rainstorm v4.0.0: one 64-byte message block, seed zero, two message rounds, no padding compression, and the unchanged fold and output finalization. A uniquely programmable first round produces a family of distinct messages indexed by one 64-bit word. On this family five folded words vanish identically, and equality of a sixth word implies equality of the complete 256-bit digest. We prove this implication, including preservation through finalization, and exhibit a concrete collision found using 4,070,333,044 evaluations of the resulting 64-bit map, plus 33,707 replay evaluations. This is an observed cost; an expected birthday-scale search cost requires a random-mapping heuristic. One restored padding round preserves five fold equalities but introduces an additional aggregate condition. The result identifies a specific interaction between message programming, cross-half XOR, and subtractive folding. It does not give a collision attack on production Rainstorm, which uses four message rounds and four padding rounds. We include the complete restricted-target definition and an executable witness verifier.

Keywords: Rainstorm; reduced-round cryptanalysis; collision attacks; subtractive folding.

1 Introduction

Rainstorm v4.0.0 is a word-oriented hash with sixteen 64-bit state words and 64-byte input blocks [1]. For a complete 64-byte message, it executes four alternating message rounds, four rounds on a mandatory padding block, subtracts the high state half from the low half, and applies width-dependent finalization. We study the effect of keeping only the first two message rounds and omitting padding compression. This is a substantial reduction of the production schedule.

The contribution is an exact structural observation about this target. The message words can program all eight transformed outputs of the first right round. Choosing those outputs as $(0, 0, 0, 0, 0, 0, v, -v)$ makes five words of the subsequent subtractive fold identically zero. A collision in the remaining first fold word then gives a complete reduced 256-bit collision. The new object of analysis is this interaction between the round and fold; the collision-search technique is standard. The argument requires no chosen initial state, probabilistic differential trail, or solver assumption.

The paper separates three kinds of evidence: a proved reduction from collisions in a 64-bit map to reduced digest collisions; one verified collision witness with recorded search cost; and an exact condition explaining why that witness does not survive a restored padding round. The reduction alone neither proves that the map behaves randomly nor supplies a general expected running time. No quantitative production security margin follows from these results.

Relation to existing techniques. Programming intermediate words is a form of message modification. The subsequent search exploits a low-dimensional image of the fold. Rebound cryptanalysis uses internal degrees of freedom and outward propagation to obtain attacks [2]; the attack here requires no inbound connection between independently chosen internal states. We therefore describe it as a fold collapse rather than a completed rebound attack. We claim a construction-specific identity, not a new generic collision-search algorithm or an exhaustive priority result over all Rainstorm analyses.

2 Complete Definition of the Restricted Target

All additions and subtractions are modulo 2^{64}, in $\mathbb{Z}/2^{64}\mathbb{Z}$. The operations $\oplus$ and & are bitwise XOR and AND. ROTR_z rotates a 64-bit word right by z bits; ROTL_z is its inverse. Words are indexed from zero and message and digest words use little-endian byte order.

The domain consists of exactly 64-byte messages M, parsed as eight words $d_0, \ldots, d_7$. The seed is fixed to zero. Initialize $h_i = 64 + p_i$, where

$$(p_0, \ldots, p_{15}) = (1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47).$$

Write $L_i = h_i$ and $H_i = h_{8+i}$. The following constants are decimal unless prefixed by `0x`:

```
K = [18446744073709551557, 13166748625691186689,
     15738366001960437 49, 14785826804856938 57,
     15816344604363663 7, 13585373498361401 51,
     28492853195207109 01, 23661571636524591 83]
Z = [17, 19, 23, 29, 31, 37, 41, 53]
C_R = 0x1032547698badcfe
C_L = 0xefcdab8967452301
P = [0x8080808080808080] * 8
```

Here P is the mandatory padding block for this message length.

Right round R_d. Set $c_0 = C_R$. In increasing lane order compute

$$q_0 = H_0, \qquad q_i = H_i - c_i \quad (i > 0), \tag{1}$$

$$y_i = \mathrm{ROTR}_{Z_i}((q_i \oplus d_i) - K_i), \qquad c_{i+1} = c_i + y_i. \tag{2}$$

The output is

$$H'_i = y_i, \qquad L'_0 = (L_0 \oplus y_0) - c_8, \qquad L'_i = L_i \oplus y_i \quad (i > 0).$$

Left round T_d. For an arbitrary incoming (L, H) set $b_0 = C_L$ and compute

$$a_0 = L_0, \qquad a_i = L_i - b_i \quad (i > 0), \tag{3}$$

$$x_i = \mathrm{ROTR}_{Z_i}((a_i \oplus d_i) - K_i), \qquad b_{i+1} = b_i + x_i. \tag{4}$$

The output is

$$L'_i = x_i, \qquad H'_0 = (H_0 \oplus x_0) - b_8, \qquad H'_i = H_i \oplus x_i \quad (i > 0).$$

Each round resets its counter. These equations specify every state word, including the cross-half final counter subtraction.

Definition 1 (Reduced digest)**.** $\mathrm{RS}_n^{(r_m, r_p)}(M)$ starts at the IV above, applies r_m rounds on d in the order $R_d, T_d, R_d, T_d, \ldots$, then r_p rounds on P starting afresh with R_P. It replaces each L_i by $F_i = L_i - H_i$, retaining H. It then applies f_n left rounds T_P, where $(f_{64}, f_{128}, f_{256}, f_{512}) = (0, 2, 4, 8)$, and emits the first $n/64$ low words in little-endian order.

Production at this length is $\mathrm{RS}_n^{(4,4)}$. The attacked function is $\mathrm{RS}_n^{(2,0)}$. Padding compression is deleted in the latter, but P is still used in finalization. A collision is a pair of distinct messages in this fixed domain with equal digests under the same target.

3 Programming and the Fold Identity

3.1 Exact right-round programming

Lemma 1 (Message programming)**.** *For every prescribed vector $(y_0, \ldots, y_7) \in \mathbb{Z}/2^{64}\mathbb{Z}^8$ and incoming state, there is a unique data block that realizes those transformed words in one right round:*

$$d_i = q_i \oplus (\mathrm{ROTL}_{Z_i}(y_i) + K_i)\,. \tag{5}$$

Proof. Invert, in order, the fixed rotation, subtraction of K_i, and XOR by d_i. The counter prefix determining q_i is known from $y_0, \ldots, y_{i-1}$. Every operation is bijective on a 64-bit word, so the recovered d_i is unique. □

This lemma converts chosen internal first-round outputs into an ordinary chosen 64-byte message. It is not a free-start relaxation.

4 Practical Two-Round Collision Attack

4.1 Fold degeneration after two rounds

Proposition 1 (Two-round fold identity)**.** *After a programmed right round and the following left round on the same data, let x_i be the transformed low words. Then $H''_0 = (y_0 \oplus x_0) - C_L - \sum_j x_j$ and $H''_i = y_i \oplus x_i$ for $i > 0$, so*

$$F_0 = C_L + \sum_{j=0}^{7} x_j - y_0 + 2(y_0 \,\&\, x_0), \tag{6}$$

$$F_i = -y_i + 2(y_i \,\&\, x_i), \qquad 1 \le i \le 7. \tag{7}$$

Proof. For $i > 0$, substitute $L_i'' = x_i$ and $H_i'' = y_i \oplus x_i$, then use the integer identity $a \oplus b = a + b - 2(a \,\&\, b)$. Word zero includes the final left-counter subtraction in H_0'', adding $C_L + \sum_j x_j$ to the fold expression. □

Choose

$$y(v) = (0, 0, 0, 0, 0, 0, v, -v). \tag{8}$$

The zero sum keeps the first-right-round wrap fixed. Proposition 1 gives $F_1 = \cdots = F_5 = 0$. Define

$$G(v) = F_0(v) = C_L + \sum_{j=0}^{7} x_j(v). \tag{9}$$

Corollary 1 (Six-word collapse)**.** *If $v_a \neq v_b$ and $G(v_a) = G(v_b)$, then the corresponding legal 64-byte messages have equal fold words $F_0, \ldots, F_5$ after two message rounds.*

Lemma 2 (Finalization preserves prefix equality)**.** *For fixed data, the first k low output words of a left round depend only on the first k low input words. The induced map on this prefix is bijective.*

Proof. The recurrence for x_0 uses only L_0. For $i > 0$, the recurrence uses L_i and the already computed $x_0, \ldots, x_{i-1}$. No high word is read. Conversely, recover $L_0 = (\mathrm{ROTL}_{Z_0}(x_0) + K_0) \oplus d_0$ and, for $i > 0$, recover $L_i = ((\mathrm{ROTL}_{Z_i}(x_i) + K_i) \oplus d_i) + C_L + \sum_{j<i} x_j$. This gives an inverse for every prefix. □

Proposition 2 (Collision reduction)**.** *Distinct v_a, v_b with $G(v_a) = G(v_b)$ yield distinct 64-byte messages colliding under each of $\mathrm{RS}_{64}^{(2,0)}$, $\mathrm{RS}_{128}^{(2,0)}$, and $\mathrm{RS}_{256}^{(2,0)}$. Their first six output words also agree under $\mathrm{RS}_{512}^{(2,0)}$.*

Proof. The vectors $y(v_a)$ and $y(v_b)$ differ in lane six. By unique programming they cannot arise from the same message at the fixed IV. The fold identity gives equality of words zero through five. Repeated application of the prefix lemma preserves this equality for any number of final left rounds on P, covering the stated output widths. No equality of the final two words is required or claimed. □

4.2 Collision algorithm and complexity

The native implementation evaluates G and interleaves 64 distinguished-point walks in a single process. This is not a claim of 64 hardware threads. A word is distinguished when its low 16 bits are zero. Starts are generated by SplitMix64 with seed 20260908; a walk is restarted after $20 \cdot 2^{16}$ steps if it has no endpoint. On a repeated endpoint, the two walks are aligned by length and replayed to find distinct predecessors with equal images. Same-path merges are discarded. The actual fold and digests are checked before reporting a collision.

Listing 1: Collision-search outline; all steps evaluate the defined map G.

```
initialize 64 walks with pseudorandom starts
repeat, interleaving one step of each walk:
    current <- G(current); length <- length + 1
    if low_16_bits(current) == 0:
        if endpoint was previously stored:
            align and replay the two walks
            if distinct predecessors a,b satisfy G(a) == G(b):
                verify the programmed messages and return them
        store endpoint, start, and length; restart walk
    else if length == 20 * 2^16:
        restart walk
```

For comparison, if t distinct inputs to a random 64-bit function have independent uniform images, then

$$\Pr[\text{a collision}] \approx 1 - \exp\left(-\frac{t(t-1)}{2 \cdot 2^{64}}\right).$$

This gives a scale of 2^{32} evaluations. Applying that scale to iterated G and its distinguished-point search is a heuristic: we do not prove random-mapping behavior, a success distribution, or an expected time bound for this specific map. The exact witness establishes that G is not injective. One successful run establishes the cost below, not a general $2^{31.9}$ attack complexity.

A G evaluation programs a message and computes two rounds and one fold word; it is not a full production hash evaluation. The recorded walk and replay counts total 4,070,366,751 evaluations, excluding initialization, table operations, and final verification. The reported throughput divides walk evaluations by recorded elapsed time. The table is historical run accounting, not a benchmark repeated for this revision. There were no recorded same-path

Table 1: Exact successful-search accounting on an Apple M1 MacBook Air.

Walk evaluations	4,070,333,044 ($2^{31.9225}$)
Replay evaluations	33,707
Stored endpoints	62,738
Endpoint merges	1
Wall time	73.572 s
Throughput	55,324,465 evaluations/s

merges or abandoned chains. The 62,738 endpoint records are a memory-use count; allocator overhead and peak resident memory were not measured.

A random 256-bit function has a generic birthday scale of 2^{128} queries. On a domain of only 2^{64} messages, it would be exceedingly unlikely to have any collision at all: the expected number is approximately 2^{-129}. Thus the demonstrated collision on this structured domain is substantive. The approximately 96-bit difference in birthday exponents is a comparison of target dimensions, not a calibrated speedup in production hash evaluations or a bound on production security.

4.3 Witness and independent verification

The recovered parameters are shown as little-endian byte strings:

```
v_a = 685a913fd5b0d24c
v_b = edf1c79141fbc6c4
```

Equation (5) yields the distinct messages:

```
message_a =
96ffffffffffffff58b56a3e941b7459bab074c685cf1afa
c02a2056945049fbfad0d3accbbf31fa1c4d50f00cd517fd
193863ec0929f2a58af12b6ca5571671

message_b =
96ffffffffffffff58b56a3e941b7459bab074c685cf1afa
c02a2056945049fbfad0d3accbbf31fa1c4d50f00cd517fd
75d7ff14f926a35834c0702bc01e3bc8
```

The integer-model fold prefixes are

$$F_a[0..5] = F_b[0..5] = (\texttt{f65354081cfe534e}, 0, 0, 0, 0, 0).$$

The first word emits as little-endian bytes `4e53fe1c085453f6`.

Table 2: Independently replayed reduced digests.

Width	Common digest
64	`4e53fe1c085453f6`
128	`62b5f41d42757d67c8a1b25310889a9d`
256	`303dbf39c7cd3ae7893f50fcc1328919`
	`25347bdd65f99d45d9a4f38c92d87b0d`

The search used the native round implementation in its explicitly reduced mapping. A separate Python integer model replays the pair, and a C++ bridge checks the production outputs. Appendix A supplies a self-contained Python verifier requiring no repository imports; it derives the messages from the parameters and checks the fold prefix, reduced digests, and padding obstruction. These are separate computational paths, not independent human reviews.

Native C++ production Rainstorm-128 replay gives distinct outputs:

```
message_a -> d26309b4c14f18dbeab1510045114504
message_b -> a53a7f6a61e35a2fc3675c1c726dd017
```

5 Extending the Attack: The Padding Boundary

Let $u_i(v)$ be the transformed high words of the next, fixed-data right round, and let $S(v) = \sum_i u_i(v)$. The two-round reduction omits this round. The following observation gives an exact, less artificial boundary.

Proposition 3 (One-padding-round extension)**.** *For the family in Equation (8), if $G(v_a) = G(v_b)$ then, after one genuine `0x80` padding right round,*

$$F_i(v_a) = F_i(v_b), \qquad 1 \le i \le 5.$$

Fold word zero is also equal if and only if $S(v_a) = S(v_b)$.

Proof. Before padding, $x_0, \ldots, x_5$ are independent of v: the programmed data words zero through five are constant, and the zero sum of y fixes L'_0, so the left-round recurrence fixes this entire prefix. For $1 \le i \le 5$, $H_i = x_i$; moreover $H_0 = x_0 - G(v)$ follows directly from the left-round equation. Equality of G therefore makes both state halves equal through lane five. The padding right round consequently has identical $u_0, \ldots, u_5$ and identical low/high words 1–5. Its final lane wraps the complete counter into low word zero, changing it by $S(v)$; high word zero remains u_0. Subtractive folding proves both statements. □

The concrete G-collision witness in Section 4.3 verifies the first statement: its post-padding fold words 1–5 are equal, while word zero differs because

$$S(v_a) = \texttt{56ddbed97c83ad47} \neq S(v_b) = \texttt{ae1525395140b829}.$$

The aggregate equality $S(v_a) = S(v_b)$ is an additional equation, not a proved independent 64-bit random condition. Enforcing equality of (G, S) is sufficient for the six-word prefix; conditional on equality of G, equality of S is necessary and sufficient. Its domain still has only 2^{64} parameters. Under a random 128-bit-map model, even enumeration of that entire domain gives only about $1/2$ expected colliding pairs and probability about $1 - e^{-1/2}$ of at least one. Consequently an unrestricted 2^{64} birthday-search claim is not justified for this fixed family, and an iteration on 128-bit outputs cannot directly serve as its 64-bit parameter walk. A useful extension would need exploitable dependence or a larger controllable message family. We give no collision or complexity bound for $\mathrm{RS}_n^{(2,1)}$.

The theorem also does not address the restored third and fourth message rounds. In particular, equality of the fold discards information about the two halves; it need not imply equal internal states. Production retains those states through further mixing before folding. The reduced witness below fails the production check. This is a limitation of this construction and witness, not a lower bound against other attacks.

6 Reproducibility and Limitations

The existing archive rainstorm-v4-cryptanalysis-eprint-v1 identifies the original campaign, not this revised manuscript. The reference file `src/rainstorm.cpp` has SHA-256

```
8b70ceb620aaaf19859733705fa39c0ce849340a701e05fb89cdc2e07bf3712f
```

The historical search record and replay record contain the messages and accounting used here. The native search source is `rho64.cpp`; the separate model is `rainstorm_model.py`. The original tag remains fixed. This revision does not claim that its manuscript or new standalone verifier is already in that archive.

The verifier in the appendix can be copied to a file and run with Python 3. It checks the witness quickly without repeating the search. In the revised working tree it is also provided as research/differential/verify_paper_collision.py. To run it or repeat the native search from the repository root:

```
python3 research/differential/verify_paper_collision.py
c++ -std=c++17 -O3 research/differential/rho64.cpp -o /tmp/rain-rho
/tmp/rain-rho search-two-round 5000000000 16 64 20260908 6 7
```

The second command sequence runs billions of evaluations. Platform-dependent timings need not match the historical Apple M1 MacBook Air (eight cores, 8 GB RAM). The mathematical reduction and witness do not depend on the reported timing.

The earlier campaign also contains internal differential measurements, backward-state searches, solver timeouts, and output screens. They are retained in the archive but are not premises of this paper's theorem. In particular, unsuccessful search is not evidence for a production security level. No production collision, preimage attack, or security proof is established here. The unresolved problem is to retain useful message control while satisfying the conditions imposed by the omitted rounds.

Research disclosure. The author designed Rainstorm. OpenAI Daybreak assisted the original campaign with hypotheses, experiment design, implementation, and analysis; AI assistance also contributed to manuscript revision. The human author is responsible for reviewing the mathematics, evidence, and final submission. The claims rest on the displayed proofs and reproducible witness, not on the research tool's authority.

7 Conclusion

For a fixed-IV, two-message-round variant without padding compression, first-round programming and subtractive folding reduce a 256-bit collision target to equality of one 64-bit map. Prefix preservation proves that the equality survives the unchanged finalization. The explicit witness establishes a practical reduced collision, while the search cost remains an observation supported by a birthday heuristic rather than a proved expected bound. One restored padding round adds an exact aggregate condition that the witness fails. Extending the attack to production remains open.

References

[1] Cris. Rainstorm v4.0.0: reference implementation. Source in rainstorm-v4-cryptanalysis-eprint-v1, 2026.

[2] M. Lamberger, F. Mendel, C. Rechberger, V. Rijmen, and M. Schläffer. The Rebound Attack and Subspace Distinguishers: Application to Whirlpool. Cryptology ePrint Archive, 2010/198, 2010. https://eprint.iacr.org/2010/198

A Executable Witness Verifier

This listing is complete Python 3 code using only built-ins. All hexadecimal parameters below are integers; the main text gives their little-endian encoding. Assertions check fixed expected values, not merely that two implementations agree. Run without Python's `-O` option, which disables assertions.

```
#!/usr/bin/env python3
"""Standalone verifier for the paper's seed-zero, 64-byte collision."""
MASK = (1 << 64) - 1
K = [18446744073709551557, 13166748625691186689,
     15738366001960437 49, 14785826804856938 57,
     15841634460436366 37, 13585373498361401 51,
     28492853195207109 01, 23661571636524591 83]
Z = [17, 19, 23, 29, 31, 37, 41, 53]
CR, CL = 0x1032547698badcfe, 0xefcdab8967452301
IV = [64 + p for p in [1, 2, 3, 5, 7, 11, 13, 17,
                       19, 23, 29, 31, 37, 41, 43, 47]]
PAD = [0x8080808080808080] * 8


def rotate(x, z):
    return ((x >> z) | (x << (64 - z))) & MASK


def round_state(h, d, left):
    # Direct implementation of the displayed two-half recurrences.
    low, high = h[:8], h[8:]
    active, other = (low, high) if left else (high, low)
    counter = CL if left else CR
    out = []
    for i in range(8):
        q = (active[i] - (counter if i else 0)) & MASK
        out.append(rotate(((q ^ d[i]) - K[i]) & MASK, Z[i]))
        counter = (counter + out[i]) & MASK
    passive = [other[i] ^ out[i] for i in range(8)]
    passive[0] = (passive[0] - counter) & MASK
    return out + passive if left else passive + out


def program(v):
    y, d, counter = [0] * 6 + [v, (-v) & MASK], [], CR
    for i in range(8):
        q = (IV[8 + i] - (counter if i else 0)) & MASK
        d.append(q ^ ((rotate(y[i], 64 - Z[i]) + K[i]) & MASK))
        counter = (counter + y[i]) & MASK
    assert round_state(IV, d, False)[8:] == y
    return d


def state(d, rm=2, rp=0):
    h = IV[:]
    for data, rounds in [(d, rm), (PAD, rp)]:
        for r in range(rounds):
            h = round_state(h, data, bool(r & 1))
    return h


def fold(h):
    return [(h[i] - h[8 + i]) & MASK for i in range(8)]


def encode(words):
    return b''.join(w.to_bytes(8, 'little') for w in words).hex()


def digest(h, bits):
    h = fold(h) + h[8:]
    for _ in range(0 if bits == 64 else max(bits // 64, 2)):
        h = round_state(h, PAD, True)
    return encode(h[:bits // 64])
```

Witness checks (continued).

```
def verify():
    da, db = program(0x4cd2b0d53f915a68), program(0xc4c6fb4191c7f1ed)
    prefix = ('96ffffffffffffff58b56a3e941b7459bab074c685cf1afa'
              'c02a2056945049fbfad0d3accbbf31fa1c4d50f00cd517fd')
    assert encode(da) == prefix + '193863ec0929f2a58af12b6ca5571671'
    assert encode(db) == prefix + '75d7ff14f926a35834c0702bc01e3bc8'
    assert da != db
    a, b = state(da), state(db)
    assert fold(a)[:6] == fold(b)[:6] == [0xf65354081cfe534e] + [0]*5
    expected = {64: '4e53fe1c085453f6',
                128: '62b5f41d42757d67c8a1b25310889a9d',
                256: ('303dbf39c7cd3ae7893f50fcc1328919'
                      '25347bdd65f99d45d9a4f38c92d87b0d')}
    for bits, value in expected.items():
        assert digest(a, bits) == digest(b, bits) == value
    assert digest(a, 512)[:96] == digest(b, 512)[:96]
    assert digest(a, 512) != digest(b, 512)
    pa, pb = state(da, 2, 1), state(db, 2, 1)
    assert fold(pa)[1:6] == fold(pb)[1:6]
    assert fold(pa)[0] != fold(pb)[0]
    assert sum(pa[8:]) & MASK == 0x56ddbed97c83ad47
    assert sum(pb[8:]) & MASK == 0xae1525395140b829
    assert digest(pa, 256) != digest(pb, 256)
    for bits in (128, 256):
        assert digest(state(da, 4, 4), bits) != digest(state(db, 4, 4), bits)
    print('PASS: messages, fold, reduced digests, padding, production checks')


if __name__ == '__main__':
    verify()
```